

ASTROSURFER – v3.02

By Griffin Shea

INTRODUCTION AND PROJECT HISTORY

Astrosurfer began as an honours project I worked on in University in 2022 but did not complete. I dropped the course and decided to pursue other 4000-level classes in its place. The original, incomplete report for that project is available as-is in this GitHub repository titled `incomplete_report.pdf`.

I continued to develop the project afterwards in my spare time and I intend to make a few more improvements in the near future. This document is intended as a guide to the codebase and an introduction to its architectural design. It will be a work in progress so expect incompleteness. I feel that it may provide a good reference for novice programmers interested in learning to write code for real-time 3D games. If you're interested in this project, feel free to contact me at griffin.shea@danjo.ca with any questions or suggestions you may have.

DEPENDENCIES

This project makes use of six external libraries. They are:

- PyGame – opens the game window and collects user input (<https://www.pygame.org/>)
- PyOpenGL – 3D rendering using the GPU (<https://mcfletch.github.io/pyopengl/documentation/index.html>)
- PyGLM – for vector, matrix, and quaternion mathematical operations (<https://pypi.org/project/pyglm/>)
- ~~Numpy – used in Renderer and ResourceManager classes to load data into OpenGL (<https://pypi.org/project/numpy/>)~~
- Attr – enables simpler definitions for game data properties (see Props below) (<https://www.attrs.org/en/stable/>)
- TraceMalloc – to track memory and other debugging purposes (<https://docs.python.org/3/library/tracemalloc.html>)

FILE STRUCTURE

Config, constants:

- Imports several libraries and defines global settings and constants used by the rest of the engine code.

Glmh:

- A collection of mathematical functions building off of the glm library.

Assets:

- contains all the fonts, meshes (3D models, as obj files), textures (images), and shader files (GLSL programs), needed for rendering the game. Also holds level files for building and running levels and scripts for game objects.

Main:

- Main function to start the program.

Controller:

- A static class that collects user input each frame using the pygame library.

Renderer:

- Class used to render each frame using OpenGL and a pygame window.

ResourceManager:

- Loads and stores asset data on the graphics card using OpenGL.

ShaderHelper:

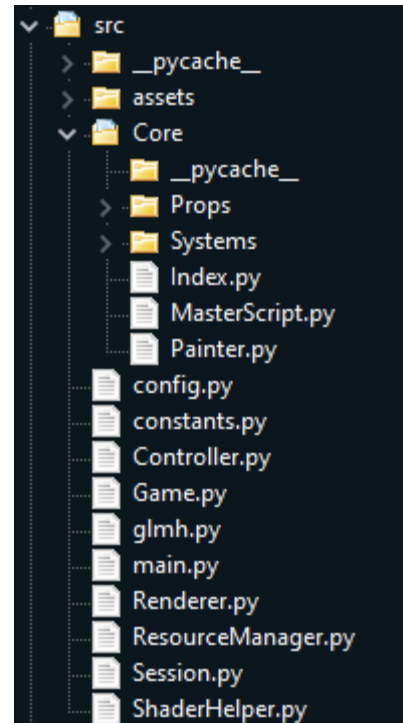
- A class used in ResourceManager to compile GLSL shader programs.

Game:

- Handles a basic menu and wraps an instance of Session

Session:

- Responsible for the main functions of running the game loop. Uses the files in Core. Uses MasterScript and Painter classes to update and render the game data. The code in Systems is used by MasterScript. Game data is contained by Index, which uses a dictionary to hold game objects defined by a key. Each game object is a combination of properties from the Props folder.



MAIN - GAME LOOP, CONTROLLER, RENDERER, AND GAME

A game loop is implemented following the pattern: input-update-draw-render. The input and render steps are handled by the static classes Controller and Renderer, while the update and draw steps are handled by an instance of Game which contains logic for a simple menu as well as level data.

```
1  from config import *
2  from Renderer import Renderer
3  from Controller import Controller
4  from Game import Game
5
6  #COMMENTS
7
8  def main():
9      random.seed(1250) #random.seed(time.time())
10     Renderer.init()
11     game = Game()
12     print("\nStarting game loop...")
13     print("=====\n")
14     while game.state != 0:      #game.state != QUIT
15         Controller.pollInput()
16         game.update()
17         game.render()
18     print("\n=====")
19     print("Game loop exit.")
20     return
21
22 if __name__ == "__main__":
23     main()
24
25 exit()
26
```

Controller is a static class with two main responsibilities: capture user input and store it in an accessible place. Controller.pollInput() records keypresses and mouse movements using the pygame library and processes them into a series of dictionaries to be accessed later using “getter” functions in Game.update().

Renderer is a static class that handles the game window, clock, and drawing functions used in Game.draw(). It must be initialized by Renderer.init() which uses pygame to open a window and then sets up an OpenGL frame buffer. It also starts a clock and loads resources using another static class called ResourceManager (ResourceManager also uses the ShaderHelper class). Renderer’s drawing functions write to the frame buffer between frames and then the frame buffer is drawn to the pygame window by Renderer.flipDisplay(), which also limits the frame rate.

GAME AND SESSION – update and render

Game is responsible for wrapping an instance of Session, which controls the game/simulation/level data, and implementing a basic menu (starting the game, quitting, pausing, and restarting).

When you run main.py and press RETURN, a Session object is instantiated in Game with a LevelFile. A LevelFile is a class stored in assets/levels with three static references (Builder, MainScript, Canvas) which are passed to Session. Session creates an Index and populates it using the builder.build() method.

```
class Session:
    def __init__(self, LevelFile):
        print("Building level:")
        self.builder = LevelFile.builder
        self.mainScript = LevelFile.mainScript
        self.canvas = LevelFile.canvas

        print("\tPopulating index... ")
        self.index = Index()
        self.builder.build(self.index)
        print("Finished.\n")
```

It also creates an empty Index to contain all of the game data, which is organized as objects: a collection of properties (called props) with a shared key. There are two main ways to access objs within the index.

- Index.get(key) returns an Obj object with that key and all props associated with it.
- Index[type] returns a list of all props of the given type.
- Index[type1, type2, etc.] returns a list of “partial objects”: tuples of key-matched props of type1, type2, etc.. This function implements a dependency requirement by which all objects with a prop of type1 must have a prop of type2, type3, etc..
- Index.match(type1, type2, ...) returns a list of tuples of props with all given types, with no dependency requirement.

The session begins to update immediately unless ESCAPE is pressed to “pause” Game. When Game is “paused” the session is drawn with a filter. In the update function of session, there is a second “pause” activated by pressing F which simply freezes the game with no filter and allows you to press L to see one frame run.

Session update then calls the MainScript set by the LevelFile and then proceeds to run the MasterScript:

```
self.mainScript.run(self, self.index)
MasterScript.run(self.index, Renderer.dTime)
```

A MainScript controls the game first to implement user input and any other high-level logic needed to run the game including changing levels, i.e., changing the builder, mainScript, and canvas of the session.

MasterScript is the *meat and potatoes* of the engine including ten discrete steps:

```
11 class MasterScript:
12     @classmethod
13     def run(cls, index, dTime):
14         cls.updateTimers(index, dTime)
15         cls.runScripts(PreScript, index)
16         Integrator.run(index, dTime)
17         cls.runScripts(InterScript, index)
18         collisions = Detector.detect(index)
19
20         #precollide scripts
21         for (keyA, keyB, contactA, contactB, sepvec) in collisions:
22
23             Solver.run(index, collisions)
24
25             #postcollide scripts
26             for (keyA, keyB, contactA, contactB, sepvec) in collisions:
27
28                 cls.runScripts(PostScript, index)
29
30                 #run delete scripts and remove deleted transfs from the index
31                 deletedKeys = [index.deleteObj(k) for (t, k) in index[Transf, "Key"] if t.delete]
32                 if deletedKeys:
33                     print("Deleted transfs: ", deletedKeys)
34
35         return
```

1. Update Timers in the index.
2. *Execute PreScripts in the index.
3. Run the Physics Integrator.
4. *Execute InterScripts in the index.
5. Run the Collision Detector, returning a list of collision artefacts.
6. *Execute Precollide scripts for each object involved in each collision.
7. Run the Physics Solver.
8. *Execute Postcollide scripts for each object involved in each collision.
9. *Execute PostScripts in the index.
10. Remove any deleted keys from the index.

Steps marked with an asterisk execute scripts written and added to objects as props.

The draw step is implemented by a call to the Painter class which draws all game objects in the index and then calls the Canvas from the LevelFile to add any final touches and draw the graphical user interface. Painter uses the Renderer to perform these functions.

REFLECTION – ROOM FOR IMPROVEMENT

- While the property-centric architecture makes parallelization easy this feature has not yet been implemented. Costly physics calculations are made on the CPU rather than in batches on the GPU. CUDA appears to be the obvious tool for this.
- Collision detection is not continuous. Objects that are fast enough can pass through brick walls as if they weren't there.
- Collision artefact is a separation vector and two contact points. Contact manifolds (surfaces) would be ideal and make for more accurate calculations.
- Constraint solver is iterative – weak. It should work on the GPU and solve constraints in groups based on distance chunking.